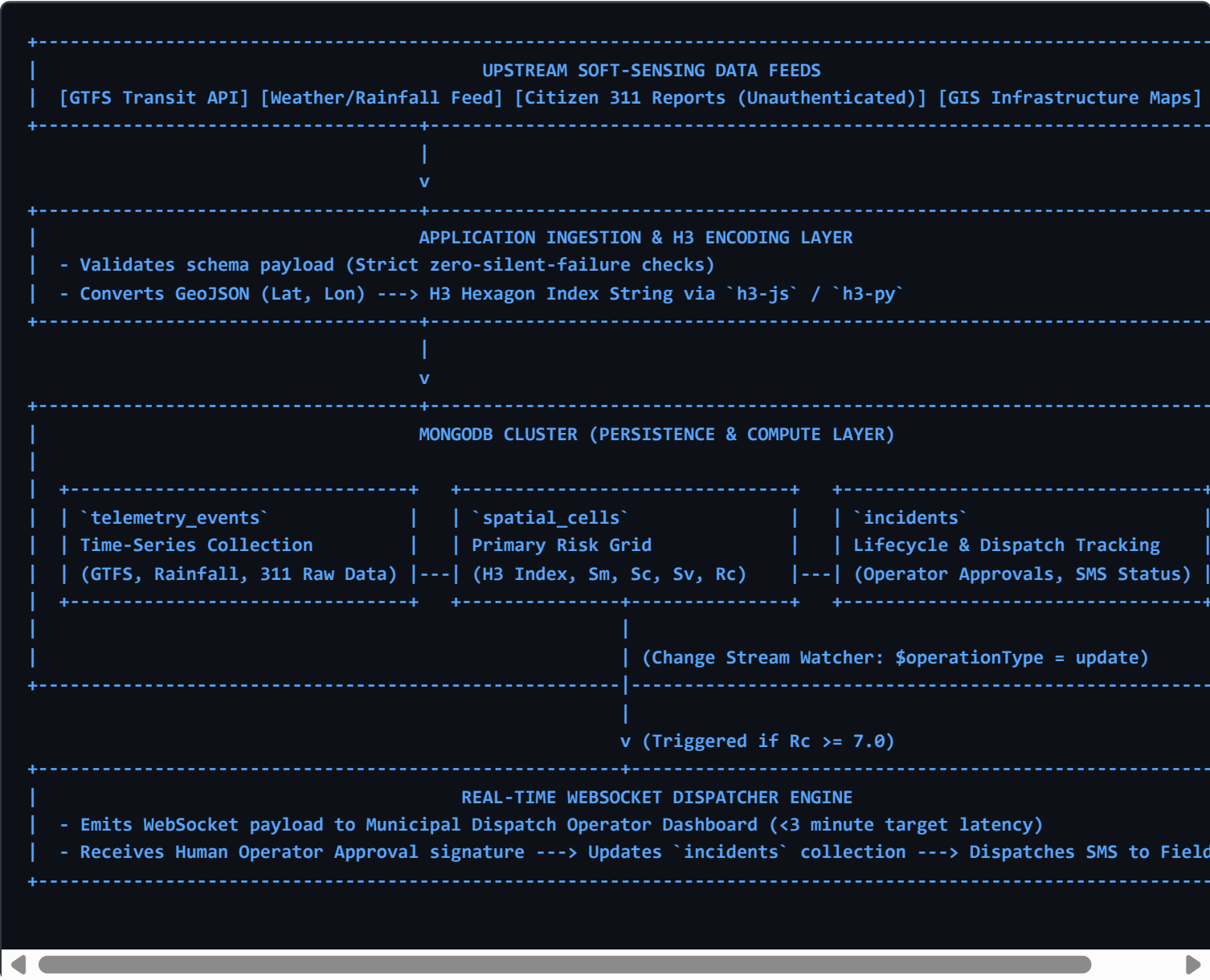


MetabolicCity AI: MongoDB Data Layer Implementation

Complete Database Schemas, Change Stream Engine, Spatial Indexing, and 4-Path Execution Pipeline

1. System Architecture & MongoDB Integration Pipeline

MetabolicCity AI utilizes MongoDB as its primary persistence and real-time operational database[cite: 8]. The system ingests heterogeneous "soft-sensing" feeds (GTFS transit, rainfall telemetry, citizen reports, GIS static maps), indexes them to spatial Uber H3 grid cells, calculates composite risk deterministically, and triggers real-time alerts when $R_c \geq 7.0$ [cite: 8].



2. Production MongoDB Schemas

A. Collection: spatial_cells (Primary Risk Grid)

Stores the continuous spatial risk state for each Uber H3 hexagonal grid cell[cite: 8]. Provides deterministic factor scores (\$S_m\$, \$S_c\$, \$S_v\$) and composite score (\$R_c\$)[cite: 8].

```
{
  "_id": ObjectId("66d21e8f9b1d2c3f4e5a6b7c"),
  "h3_index": "8826856235fffff",
  "resolution": 8,
  "location": {
    "type": "Point",
    "coordinates": [77.3150, 23.1812] // [Longitude, Latitude]
  },
  "scores": {
    "mobility": 8.1,           // Sm: Transit delays, stoppages (0.0 - 10.0)
    "climate": 7.2,           // Sc: Rainfall, elevation, weather (0.0 - 10.0)
    "vulnerability": 6.8      // Sv: Hospitals, underpasses, schools (0.0 - 10.0)
  },
  "weights": {
    "Wm": 0.35,               // Mobility Weight
    "Wc": 0.40,               // Climate Weight (Configurable seasonally)
    "Wv": 0.25                // Vulnerability Weight
  },
  "composite_score": 7.415, // Rc = (Wm * Sm) + (Wc * Sc) + (Wv * Sv)
  "status": "CRITICAL",      // "NORMAL" | "ELEVATED" | "CRITICAL"
  "last_updated": ISODate("2026-08-30T19:30:00.000Z")
}
```

B. Collection: telemetry_events (Time-Series Ingestion)

Time-series collection configured to log raw soft-sensing data feeds for auditability and risk score recalculation[cite: 8].

```
// Collection Created via: db.createCollection("telemetry_events", { timeseries: { timeField: "timestamp", metaField: "timestamp" } })
{
  "_id": ObjectId("66d21e8f9b1d2c3f4e5a6b7d"),
  "timestamp": ISODate("2026-08-30T19:29:55.000Z"),
  "source_type": "CITIZEN_311", // "GTFS_TRANSIT" | "WEATHER_API" | "CITIZEN_311" | "GIS_STATIC"
  "h3_index": "8826856235fffff",
  "payload": {
    "category": "DRAINAGE_BLOCKAGE",
    "severity_index": 8.5,
    "reporter_channel": "UNAUTHENTICATED_WEB_FORM",
    "raw_coordinates": { "lat": 23.1812, "lng": 77.3150 }
  }
}
```

C. Collection: incidents (Workflow & Dispatch State)

Tracks critical incidents (\$R_c \ge 7.0\$) throughout their lifecycle: detection, operator approval, SMS field dispatch, and resolution[cite: 8].

```
{
  "_id": ObjectId("66d21e8f9b1d2c3f4e5a6b7e"),
  "incident_id": "INC-20260830-0089",
  "h3_index": "8826856235fffff",
  "trigger_score": 7.415,
  "score_breakdown": { "Sm": 8.1, "Sc": 7.2, "Sv": 6.8 },
}
```

```
"recommended_action": "Deploy drainage unit and reroute bus route #14 away from underpass.",
"approval_workflow": {
  "status": "APPROVED", // "PENDING_APPROVAL" | "APPROVED" | "REJECTED" | "RESOLVED"
  "operator_id": "OP-9942",
  "decision_timestamp": ISODate("2026-08-30T19:31:10.000Z")
},
"dispatch_details": {
  "field_crew_id": "CREW-DRAINAGE-04",
  "notification_channel": "SMS",
  "sms_status": "DELIVERED",
  "sent_at": ISODate("2026-08-30T19:31:12.000Z")
},
"metrics": {
  "detection_latency_seconds": 120, // Measured from telemetry timestamp to alert emission (<3 mins target)
  "dispatch_latency_seconds": 62,   // Measured from alert emission to SMS dispatch
  "resolution_timestamp": null
}
}
```

3. Trace 4 Execution Paths (Data Ingestion to Risk Update)

To eliminate silent failures, every input ingestion flow into MongoDB must account for 4 distinct execution paths:

EXECUTION PATH	SYSTEM CONDITION	MONGODB ARCHITECTURAL BEHAVIOR
1. Happy Path	Valid telemetry coordinates, non-zero risk metrics, active H3 cell matching[cite: 8].	Writes event to telemetry_events. Recalculates \$R_c\$. Executes atomic \$set on spatial_cells. Change Stream emits WebSocket alert if \$R_c \ge 7.0\$[cite: 8].
2. Nil / Null Input Path	Ingestion payload missing coordinates, invalid null metrics, or unresolvable H3 mapping.	Explicit Exception raised (InvalidPayloadException). Payload rejected before DB update. Incident logged to error-sink collection with schema validation details. Zero silent dropping.
3. Empty Input Path	Valid payload schema but empty payload values (e.g., rainfall telemetry reading 0.0 mm/hr delta).	Telemetry logged to telemetry_events. Recalculation logic skips database update to spatial_cells if factor score delta \$< 0.01\$, preventing unnecessary DB writes and lock contention.
4. Upstream System Error Path	MongoDB network partition, primary write timeout, or Change Stream connection drops.	Ingestion layer falls back to local in-memory queue (Redis buffer). Retries database write with exponential backoff. System status flagged elevated on dispatcher dashboard.

4. Core Operations: Atomic Risk Scoring & Change Streams

Deterministic Risk Calculation Pipeline

$$R_c = (W_m \times S_m) + (W_c \times S_c) + (W_v \times S_v)$$

The following Node.js / MongoDB update pipeline atomically recalculates factor scores and updates the Composite Risk Score (\$R_c\$) in a single atomic database operation[cite: 8]:

```
async function updateCellRiskScore(db, h3Index, newSm, newSc) {
  if (!h3Index || newSm == null || newSc == null) {
    throw new Error("ERR_NIL_INPUT: h3Index, newSm, and newSc are required parameters.");
  }

  const result = await db.collection("spatial_cells").updateOne(
    { h3_index: h3Index },
    [
      {
        $set: {
          "scores.mobility": newSm,
          "scores.climate": newSc,
          "composite_score": {
            $add: [
              { $multiply: ["$weights.Wm", newSm] },
              { $multiply: ["$weights.Wc", newSc] },
              { $multiply: ["$weights.Wv", "$scores.vulnerability"] }
            ]
          },
        },
        "status": {
          $cond: {
            if: {
              $gte: [
                {
                  $add: [
                    { $multiply: ["$weights.Wm", newSm] },
                    { $multiply: ["$weights.Wc", newSc] },
                    { $multiply: ["$weights.Wv", "$scores.vulnerability"] }
                  ]
                },
                7.0
              ]
            },
            then: "CRITICAL",
            else: "NORMAL"
          }
        },
        "last_updated": new Date()
      }
    ]
  );

  if (result.matchedCount === 0) {
    throw new Error(`ERR_H3_NOT_FOUND: Spatial cell ${h3Index} does not exist in spatial_cells collection.`);
  }
}
```

Real-Time Alert Change Stream Listener (<3 Min Target Latency)

Listens continuously to updates on the `spatial_cells` collection and filters for documents where $R_c \geq 7.0$, pushing alerts to the dispatcher UI[cite: 8]:

```
function initializeRiskAlertStream(db, io) {
  const collection = db.collection("spatial_cells");

  const changeStream = collection.watch([
    {
      $match: {
        "operationType": "update",
        "fullDocument.composite_score": { $gte: 7.0 }
      }
    }
  ], { fullDocument: "updateLookup" });

  changeStream.on("change", async (change) => {
    const doc = change.fullDocument;

    // Construct Alert Payload for Municipal Dispatch Operator UI
    const alertPayload = {
      event: "CRITICAL_RISK_THRESHOLD_EXCEEDED",
      h3_index: doc.h3_index,
      composite_score: doc.composite_score,
      explanation: {
        mobility_score: doc.scores.mobility,
        climate_score: doc.scores.climate,
        vulnerability_score: doc.scores.vulnerability,
        weights: doc.weights
      },
      recommended_action: `High flood risk detected at cell ${doc.h3_index}. Reroute transit vehicles and deploy
      timestamp: doc.last_updated
    };

    // Push alert instantly via WebSocket
    io.to("role:dispatch_operator").emit("CRITICAL_ALERT", alertPayload);
  });

  changeStream.on("error", (error) => {
    // Zero silent failure: Log exception and re-initialize connection listener
    console.error("FATAL: MongoDB Change Stream failure in RiskAlertStream:", error);
    setTimeout(() => initializeRiskAlertStream(db, io), 5000);
  });
}
```



5. MongoDB Indexing Strategy & Performance Matrix

COLLECTION	INDEX DEFINITION	TYPE	OPERATIONAL PURPOSE
spatial_cells	{ h3_index: 1 }	Unique Compound	Sub-millisecond single cell lookup for risk updates[cite: 8].
spatial_cells	{ location: "2dsphere" }	Geospatial 2DSphere	Proximity queries for unmapped raw coordinates[cite: 8].
spatial_cells	{ composite_score: -1 }	Descending Single	Allows operators to sort highest-risk city grid cells instantly[cite: 8].

Collection	Index Definition	Type	Operational Purpose
telemetry_events	{ timestamp: -1, source_type: 1 }	Compound Time-Series	Fast retrieval of recent telemetry feeds for diagnostic audit logs[cite: 8].
incidents	{ "approval_workflow.status": 1, incident_id: 1 }	Compound Index	Quick filtering of pending vs approved operational dispatches[cite: 8].

6. Database Architectural Review Report

Evaluation Category	Architectural Verdict	Design Justification
Schema Elasticity	✅ APPROVED	Document model accommodates varied soft-sensing data feeds without migrations[cite: 8].
Detection Latency (<3 Min Target)	✅ APPROVED	MongoDB Change Streams eliminate database polling latency entirely[cite: 8].
Score Explainability	✅ APPROVED	Factor breakdowns (\$S_m, S_c, S_v\$) stored explicitly in single cell document[cite: 8].
Hardware Cost Impact	✅ APPROVED	Software-only data indexing avoids CapEx physical sensor investments[cite: 8].